Effect of `operator<=>` on the C++ Standard Library

Document Number: P0790R0
Date: 2017-10-06
Author: David Stone (dstone50@bloomberg.net, david@doublewise.net)
Audience: LEWG

This paper is to evaluate what changes we should make to the C++ standard library in response to http://open-std.org/JTC1/SC22/WG21/docs/papers/2017/p0515r1.pdf, which adds `operator<=>` to the language.

My general algorithm to determine what operations to support are: If the type represents a value of some sort, it should at least be weak_equality. If the type has some sort of meaningful ordering, it should have weak_ordering. We should be cautious in giving polymorphic types `operator<=>` (but if they already have other comparison operators, then we might as well).

This document should contain a complete list of types or categories of types in C++.

Backward Compatibility

The `operator<=>` proposal was written such that the "generated" operators are equivalent to source code rewrites – there is no actual `operator==` that a user could take the address of. This has the unfortunate effect that moving from the old style to the new is a breaking change, but the situations in which anything is broken is rare. I believe it is limited to users forming a pointer to a function that points at one of these comparison operators, which requires the user to know whether the function was implemented as a member or a free function. If it was implemented as a friend function and it was defined inline, users cannot take the address of that function, further limiting the scope of the breakage. There are some cases where we do not specify how the operator was implemented, only that the expression `a @ b` is valid; these cases are not broken by such a change because users could not have depended on it, anyway.

Types that should not get `operator<=>`

- exception types
- tag classes? (nothrow, piecewise_construct_t, etc.)
- arithmetic function objects (plus, minus, etc.)
- comparison function objects (equal_to, etc.)
- owner_less
- logical function objects (logical_and, etc.)
- bitwise function objects (bit_and, etc.)
- nested_exception
- allocator_traits
- char_traits
- iterator_traits
- numeric_limits
- pointer_traits
- regex_traits
- chrono::duration_values
- tuple_element
- max_align_t
- map::node_type
- map::insert_return_type
- set::node_type

- set::insert_return_type
- unordered_map::node_type
- unordered_map::insert_return_type
- unordered_set::node_type
- unordered_set::insert_return_type
- any
- default_delete
- aligned_storage
- aligned_union
- system_clock
- steady_clock
- high_resolution_clock
- locale::facet
- locale::id
- ctype_base
- ctype
- ctype_byname
- codecvt_base
- codecvt
- codecvt_byname
- num_get
- num_put
- numpunct
- numpunct_byname
- collate
- collate_byname
- time_get
- time_get_byname
- time_put
- time_put_byname
- money_base
- money_get
- money_put
- money_punct
- moneypunct_byname
- message_base
- messages
- messages_byname
- FILE
- va_list
- back_insert_iterator
- front_insert_iterator
- insert_iterator
- ostream_iterator
- ostreambuf_iterator
- ios_base
- ios_base::Init
- basic_ios
- basic_streambuf
- basic_istream
- basic_iostream
- basic_ostream
- basic_stringbuf
- basic_istringstream

- basic_ostringstream
- basic_stringstream
- basic_filebuf
- basic_ifstream
- basic_ofstream
- basic_fstream
- atomic_flag
- thread
- mutex
- recursive_mutex
- timed_mutex
- timed_recursive_mutex
- lock_guard
- scoped_lock
- unique_lock
- once_flag
- shared_mutex
- shared_timed_mutex
- shared_lock
- condition_variable
- condition_variable_any
- promise
- future
- shared_future
- packaged_task
- random_device
- hash
- weak_ptr
- basic_regex
- sequential_execution_policy
- parallel_execution_policy
- parallel_vector_execution_policy
- default_searcher?
- boyer_moore_searcher?
- boyer_moore_horspool_searcher?
- initializer_list: `initializer_list` is a reference type. It would be strange to give it reference semantics on copy but value semantics for comparison. It would also be surprising if two initializer lists containing the same set of values compared as not equal. Therefore, I recommend not defining it for this type without a separate paper advocating for one particular set of semantics.

Types that get their `operator<=>` from a conversion operator

These types would get `operator<=>` if possible already. Do we want to define an explicit operator or just rely on the conversion?

- integral_constant (has `operator T`) and all types deriving from integral_constant
- bitset::reference (has `operator bool`)
- reference_wrapper (has `operator T`)
- atomic (has `operator T`)

Types that should get `operator<=>`, no change from current comparisons

- error_category: strong_ordering
- error_code: strong_ordering
- error_condition: strong_ordering
- exception_ptr: strong_ordering
- type_info: strong_equality (see `type_index` for a strong comparison category type)
- monostate: strong_ordering
- bitset: strong_equality (should this be strong_ordering?)
- allocator: strong_equality
- unique_ptr: strong_ordering, heterogeneous with T1, D1 vs T2, D2 and nullptr_t
- shared_ptr: strong_ordering, heterogeneous with T1 vs T2 and nullptr_t
- memory_resource: strong_equality
- polymorphic_allocator: strong_equality
- synchronized_pool_resource: (from memory_resource base)
- unsynchronized_pool_resource: (from memory_resource base)
- monotonic_buffer_resource: (from memory_resource base)
- scoped_allocator_adaptor: strong_equality
- pool_options: strong_equality
- function: strong_equality with nullptr_t only (no homogenous operator)
- chrono::duration: strong_ordering, heterogeneous
- chrono::time_point: strong_ordering, heterogeneous in the duration
- type_index: strong_ordering
- basic_string: (strong|weak)_ordering (depends on CharT, heterogeneous with CharT const *)
- basic_string_view: (strong|weak)_ordering (depends on CharT)
- locale: strong_equality
- complex: (strong|weak)_equality (heterogeneous with T)
- linear_congruential_engine: strong_equality
- mersenne_twister_engine: strong_equality
- subtract_with_carry_engine: strong_equality
- discard_block_engine: strong_equality
- independent_bits_engine: strong_equality
- shuffle_order_engine: strong_equality
- seed_seq: no * (strong_equality)
- uniform_int_distribution: strong_equality
- uniform_int_distribution::param_type: strong_equality
- uniform_real_distribution: strong_equality
- uniform_real_distribution::param_type: strong_equality
- bernoulli_distribution: strong_equality
- bernoulli_distribution::param_type: strong_equality
- binomial_distribution: strong_equality
- binomial_distribution::param_type: strong_equality
- geometric_distribution: strong_equality
- geometric_distribution::param_type: strong_equality
- negative_binomial_distribution: strong_equality
- negative_binomial_distribution::param_type: strong_equality
- poisson_distribution: strong_equality
- poisson_distribution::param_type: strong_equality
- exponential_distribution: strong_equality
- exponential_distribution::param_type: strong_equality
- gamma_distribution: strong_equality
- gamma_distribution::param_type: strong_equality
- weibull_distribution: strong_equality
- weibull_distribution::param_type: strong_equality
- extreme_value_distribution: strong_equality
- extreme_value_distribution::param_type: strong_equality

- normal_distribution: strong_equality
- normal_distribution::param_type: strong_equality
- lognormal_distribution: strong_equality
- lognormal_distribution::param_type: strong_equality
- chi_squared_distribution: strong_equality
- chi_squared_distribution::param_type: strong_equality
- cauchy_distribution: strong_equality
- cauchy_distribution::param_type: strong_equality
- fisher_f_distribution: strong_equality
- fisher_f_distribution::param_type: strong_equality
- student_t_distribution: strong_equality
- student_t_distribution::param_type: strong_equality
- discrete_distribution: strong_equality
- discrete_distribution::param_type: strong_equality
- piecewsie_constant_distribution: strong_equality
- piecewsie_constant_distribution::param_type: strong_equality
- piecewise_linear_distribution: strong_equality
- piecewise_linear_distribution::param_type: strong_equality
- filesystem::path: strong_ordering
- filesystem::path::iterator: strong_ordering
- filesystem::directory_entry: strong_ordering
- filesystem::directory_iterator: strong_ordering
- filesystem::recursive_directory_iterator: strong_ordering
- istream_iterator: heterogeneous strong_equality
- istreambuf_iterator: heterogeneous strong_equality
- match_results: strong_equality
- regex_iterator: strong_equality
- regex_token_iterator: strong_equality
- thread::id: strong_ordering
- fpos: strong_equality
- array::iterator: strong_ordering
- deque::iterator: strong_ordering
- forward_list::iterator: strong_equality
- list::iterator: strong_equality
- vector::iterator: strong_ordering
- map::iterator: strong_equality
- set::iterator: strong_equality
- multimap::iterator: strong_equality
- multiset::iterator: strong_equality
- unordered_map::iterator: strong_equality
- unodered_set::iterator: strong_equality
- unordered_multimap::iterator: strong_equality
- unodered_multiset::iterator: strong_equality
- valarray::iterator: strong_ordering

Types that should get `operator<=>` and they wrap another type

These should all just return the strongest ordering supported by all types that they wrap:

- array
- deque
- forward_list
- list

- vector (including vector<bool>)
- map
- set
- multimap
- multiset
- unordered_map
- unodered_set
- unodered_multimap
- unordered_multiset
- queue
- queue::iterator
- priority_queue
- priority_queue::iterator
- stack
- stack::iterator
- pair
- tuple
- reverse_iterator
- move_iterator
- sub_match: weak_ordering heterogeneous with basic_string_view (cannot pick up from base object pair, as sub_match only compares one of the sub-objects)
- optional, including heterogeneous `operator<=>` with `optional<T> <=> optional<U>`, `optional<T> <=> U`, `optional<T> <=> nullopt_t`, and `nullopt_t <=> nullopt_t`
- variant:

```
// + 1 to make valueless_by_exception ordered lowest
if (auto cmp = (lhs.index() + 1 <=> rhs.index() + 1); cmp != 0) {
    return cmp;
}
return std::get<lhs.index()>(lhs) <=> std::get<rhs.index()>(rhs);
```

Types that should get `operator<=>` that have no comparisons now

- enable_shared_from_this: strong_ordering to allow derived classes to `= default`
- integer_sequence: strong_equality
- ratio: We should create heterogeneous `operator<=>` on values instead of being forced to use `ratio_equal` on types
- filesystem::file_status: strong_equality
- filesystem::space_info: strong_equality
- slice: strong_equality
- slice_array: strong_equality
- gslice: strong_equality
- gslice_array: strong_equality
- mask_array: strong_equality
- indirect_array: strong_equality
- to_chars_result: strong_equality
- from_chars_result: strong_equality
- nullptr_t: strong_ordering. It may be surprising that nullptr_t does not have any comparison operators already. It is comparable with many other types, and edg, gcc, and Visual Studio provide all comparison operators. clang, meanwhile, provides only `nullptr == nullptr` and `nullptr != nullptr`.

Types from C that should get `operator<=>`

- div_t, ldiv_t, lldiv_t, imaxdiv_t: strong_ordering

- timespec: strong_ordering
- tm: strong_ordering
- lconv: strong_equality
- fenv_t: strong_equality
- fpos_t: strong_ordering
- mbstate_t: strong_equality

valarray

Current comparison operators return a `valarray<bool>`, giving you the result for each pair (with undefined behavior for differently-sized `valarray` arguments). It would make sense to provide some sort of function that returns `valarray<comparison_category>`. Consistency with current `valarray` behavior suggests we name that function `operator<=>`, but consistency with `operator<=>` suggests we name it anything else.

New Functions / Function Objects

- owner_compare: by analogy with `owner_less` for `shared_ptr` and `weak_ptr`. Do we follow the pattern of `owner_less` and make the type a template, or do we just make its `operator()` a template? If we are planning on deprecating `owner_less`, I would recommend making `operator()` a template so we do not have to do the "`owner_less<>` is generic" business. Returns `strong_ordering`.
- shared_ptr::owner_compare: See above.
- weak_ptr::owner_compare: See above.

Updating Comparison Concepts

Components based on equality

All of these components require a function that tests for equality and do so via a function matching the concept `BinaryPredicate`. This is a complete set of operations that make use of the `BinaryPredicate` concept. We should extend `BinaryPredicate` to also allow a function returning `weak_equality` or better.

- find_end
- find_first_of
- adjacent_find
- mismatch
- equal
- search
- search_n
- unique
- unique_copy
- default_searcher
- boyer_moore_searcher
- boyer_moore_horspool_searcher
- UnorderedAssociativeContainer

Components based on ordering

All of these components require a strict weak ordering and do so via a function matching the concept `Compare`. This is a complete set of operations that make use of the `Compare` concept. We should extend `Compare` to also allow a function returning `weak_ordering` or better. If any of these components (for instance, `map`) are passed an

instance of `Compare` returning an ordering, it can then use a single call to the comparison function to determine equivalence, rather than relying on `!comp(lhs, rhs) and !comp(rhs, lhs)`.

- sort
- stable_sort
- partial_sort
- partial_sort_copy
- is_sorted
- is_sorted_until
- nth_element
- lower_bound
- upper_bound
- equal_range
- binary_search
- merge
- inplace_merge
- includes
- set_union
- set_intersection
- set_difference
- set_symmetric_difference
- push_heap
- pop_heap
- make_heap
- sort_heap
- is_heap
- is_heap_until
- min
- max
- minmax
- min_element
- max_element
- minmax_element
- clamp
- lexicographical_compare
- next_permutation
- prev_permutation
- forward_list::merge
- forward_list::sort
- list::merge
- list::sort
- AssociativeContainer
- priority_queue

Open Questions

When we specify generic type concepts, do we specify in terms of `operator<=>`? If we do so, it means that functions that accept a UDT corresponding to some concept no longer match that concept. As an example, the `RandomAccessIterator` concept requires all six operators to be present. At some point in the future, do we want to change this requirement to `weak_ordering operator<=>`? Same with weaker iterator concepts, do we want `weak_equality` or stronger? The following are the minimal mappings I believe each concept requires. Note that by applying `operator<=>`, we frequently end up with more operators than just those minimally required by the existing concept

- EqualityComparable: `weak_equality`
- LessThanComparable: `weak_ordering` (this would exclude floating point values, as `LessThanComparable` requires a strict weak ordering)
- NullablePointer: inherits `EqualityComparable`, additionally requires `!=` (works by default with `weak_equality`). Requires heterogeneous `weak_equality` with `nullptr_t`.
- InputIterator: inherits `EqualityComparable`, additionally requires `!=` (works by default with `weak_equality`).
- Allocator: `weak_equality`, heterogeneous with the equivalent `Allocator` for other types
- "bitmask type": strong_(equality|ordering) (it might be a bitset, which only supports equality, it might be integer / enum, which supports ordering, but this will naturally fall out, no wording changes needed)
- We redundantly specify `weak_ordering` for TrivialClock::rep, TrivialClock::duration, and TrivialClock::time_point, but that is implied because we know they are an arithmetic type, a chrono::duration, and a chrono::time_point, respectively.

Deprecation

We should deprecate or remove existing comparison operators for types that do not defer to user-defined types (for instance, `allocator`) but which get `operator<=>`.

Types that do wrap user-defined types (`vector`, `optional`, etc.) should provide only `operator<=>` if the underlying type defines `operator<=>`. If the underlying type does not define `operator<=>`, we should provide only the old operators (do we deprecate this support?). Following this guidance when the "user-defined" type is actually a standard type (`vector<string>`) or when the underlying type is a built-in type leads to a slight backward incompatibility. We could provide deprecated explicit operator overloads to prevent this.

We should deprecate existing three-way comparison functions:

- basic_string::compare
- basic_string_view::compare
- filesystem::path::compare
- sub_match::compare

We should deprecate rel_ops. We should also remove the provision that states that "In this library, whenever a declaration is provided for an operator!=, operator>, operator>=, or operator<=, and requirements and semantics are not explicitly provided, the requirements and semantics are as specified in this Clause." in reference to rel_ops.

We should not add `operator<=>` to any deprecated types.

Miscellaneous

All `operator<=>` should be constexpr noexcept where possible, following the lead of the language feature and allowing `= default` as an implementation strategy for some types.

When we list a result type as "unspecified" it is unspecified whether it has `operator<=>`. I do not believe there are any unspecified result types for which we currently guarantee any comparison operators are present.